WEEK 16

The I²C Bus

Two wires, any number of devices. Addresses in the message, acknowledgment at last, and the open-drain trick that lets everyone share one line

Textbook: Dean, *Embedded Systems Fundamentals*, 2nd ed., Chapter 8, printed pages 269–283.
Lab manual: Lab 09 (I²C with the MPU-6050 / LSM303AGR). Your own project plan.

1 What we are actually after this week

Last week's SPI was fast and simple and needed a select line for every device. This week we get rid of them.

The question: **can several devices share exactly two wires, with no select lines at all?** Yes, and the price is instructive. The plan:

1. **Two wires**, and the **open-drain** circuit that lets several devices drive one line without destroying each other.
2. **Speeds**, and why the maximum depends on your wiring rather than your chip.
3. **The message format**: start, address, R/W, acknowledgment, data, stop.
4. **Addressing**, and the shift-left-by-one that catches everybody.
5. **Write and read transactions**, including the **repeated start** that makes a register read work.
6. **Register addressing** and **auto-increment**.
7. The peripheral, and a full worked sensor example.
8. The three protocols side by side, which is the exam's favourite comparison.

Item 4 is the single commonest I²C bug. Item 5's repeated start is the concept most people cannot explain under questioning. Item 8 is where the last three weeks pay off.

2 Deep dive 1: Two wires, and open drain

I²C

> **Inter-Integrated Circuit bus**: a synchronous master/slave protocol using exactly two signals, **SDA** (serial data) and **SCL** (serial clock).
>
> The master initiates all communication and slaves transmit only when allowed to. Its defining feature is **device addressing**: every message contains an address, every device has a unique one, and **only the addressed device responds**.
>
> This is what lets many devices share one bus **without any additional control signals**, unlike SPI's select lines.

Recall Week 14's problem 6: address via separate select lines, or inside the message. SPI chose select lines; I²C chooses the message. That single decision produces almost every difference between them.

2.1 The open-drain circuit

Now a real problem. If several devices share one wire, what happens when one drives it high and another drives it low? On an ordinary push-pull output that is a direct short between supply and ground, which destroys both drivers.

Recall `OTYPER` from Week 03, where we parked open drain and said I²C would explain it. Here it is.

Open drain, and wired-AND

Each device's drive circuit is a **single transistor between the bus signal and ground**. There is no transistor pulling up. A separate **pull-up resistor** connects the line to V_{DD}.

To send a **0**: turn the transistor on, pulling the line to ground.
To send a **1**: turn the transistor **off**, and let the pull-up resistor raise the line to V_{DD}.

So no device ever actively drives high. **If two devices transmit different data simultaneously, the line reads 0**, because any transistor turning on wins over the resistor.

This is a **wired-AND**: the line is 1 only if *every* device is releasing it.

Two consequences worth drawing out, because they explain the rest of the protocol.

Collisions are harmless. Two devices transmitting at once produce a valid electrical signal, just not the one both intended. Nothing burns out. That is what makes shared-bus arbitration possible at all.

Rising edges are slow. A transistor pulling low is fast and strong. A resistor pulling high is neither, and it must charge the capacitance of every device and every centimetre of trace on the bus. Which brings us straight to speed.

Beware the trap

The pull-up resistors are not optional and they are not on your MCU.

If nobody fits them, the line can only ever be pulled low or float, so it never reaches a valid logic 1 and communication is impossible. Typical values are 2.2 kΩ to 10 kΩ.

On a Discovery board they are already fitted for the on-board sensors, which is why Lab 09 works without you thinking about it. **Wire up an external I²C device on a breadboard and you must supply them yourself.** A dead external sensor with the on-board one working perfectly is very often this.

Note also the trade: a smaller resistor charges the line faster, allowing higher speed, but wastes more current every time the line is low. That is the actual engineering choice behind the value.

2.2 Speeds

Mode	Bit rate
Standard	100 kbit/s
Full speed	400 kbit/s
Fast	1 Mbit/s
High speed	3.2 Mbit/s

Why the ceiling is a property of your board

> The maximum speed is limited by **how quickly the pull-up resistor can raise SCL or SDA to** V_{CC}, which depends on the **capacitance** of the line, which depends on **how many devices are attached and how long the bus is**.
>
> So unlike SPI, where the ceiling was a number in the slave's datasheet, an I^2C bus's real maximum is a property of your *wiring*. Add a fourth sensor or a longer cable and a bus that ran at 400 kHz may stop working, with no code change at all.
>
> Compare against SPI's tens of megahertz. **I^2C is roughly an order of magnitude slower**, and that is the price of the two-wire topology.

3 Deep dive 2: The message format

Field	Purpose
Start condition	Marks the start of a message
Slave device address	7 bits, identifies the target
R/W bit	1 = read from slave, 0 = write to slave
ACK bit	Two uses: whether the addressed slave *is present*, and whether more data will be read
Data byte(s)	For many devices the first is a **register address**
Stop condition	Marks the end
Repeated start	Optional, used in some message types

Everything is structured as a sequence of **conditions** (start, stop) and **transfers** (one byte plus one acknowledgment bit).

The acknowledgment, which UART and SPI both lacked

> Recall Week 14's problem 5. UART has no acknowledgment. SPI has no acknowledgment. **I^2C has one after every single byte.**
>
> So an I^2C master can *know* whether a device answered. If the ACK bit is not asserted after the address, **the master terminates the message with a stop condition**, because nothing at that address is listening.
>
> This is why an I^2C bus scan is possible and an SPI one is not: you can send an address to all 128 possible values and see which ones acknowledge. Recall from Week 15 that SPI's substitute was reading a known-constant register, which only works if you already know what device you are talking to.

4 Deep dive 3: Addressing, and the shift

Two modes exist, 7-bit and 10-bit. We use 7-bit.

The first byte after the start condition has two parts: the **device address in the upper seven bits**, and the **R/W bit as the LSB**.

The formula, and the bug

> $$\text{first byte} = (\text{address} \ll 1) \mid \text{R/W}$$

The byte is formed by **shifting the slave address left by one bit** and adding the R/W bit.

So a device at address `0x19` is written to with `0x32` and read from with `0x33`.

Beware the trap

This is the classic I²C bug and it exists because the industry cannot agree on what "the address" means.

Some datasheets quote the **7-bit address** (`0x19`). Others quote the **8-bit write address** (`0x32`). Others helpfully give both, or give the read address (`0x33`) and call it the address.

And HAL adds its own layer: **ST's HAL functions expect the shifted, 8-bit form**, so `HAL_I2C_Mem_Read(&hi2c1, 0x33, ...)` is correct while passing `0x19` silently addresses the wrong device.

Symptom: the device never acknowledges, so every transaction fails identically whether the address is wrong, the wiring is wrong, or the device is dead.

Your own project plan lists this exact fault: for `WHO_AM_I` returning `0x00` from the accelerometer, cause (a) is "I²C address wrong: use `0x32` for write, `0x33` for read." That is the shifted form of `0x19`, and you already found this the hard way.

4.1 Writing

Master sends: **start**, address with W(0), then data bytes, then **stop**. The slave acknowledges after the address and after *every* data byte.

4.2 Reading

Master sends: **start**, address with R(1). If the slave acknowledges, the master clocks bytes out of it. And here the ACK bit changes meaning:

- After each byte it wants to continue reading, the **master** sends an **ACK**.
- After the last byte it wants, the master sends a **NACK** (1), telling the slave to stop.
- Then the master sends the **stop** condition.

Note who is acknowledging

When *writing*, the **slave** acknowledges each byte, meaning "received".
When *reading*, the **master** acknowledges each byte, meaning "send me another".

So the final NACK before a stop is not an error. It is the master saying "that was the last one I wanted". **Failing to send the NACK on the final byte is a real bug** that leaves the slave expecting to transmit more, and it is easy to write.

5 Deep dive 4: Register addressing and the repeated start

Most sensors present themselves as a set of addressable registers, which standardises how information is organised and simplifies development. The convention: **the first data byte of a write is interpreted by the slave as a register address**.

Writing to a register is therefore straightforward: start, device address with W, register address, data, stop.

Reading from a register is not, and this is the interesting part. You must first *tell* the device which register you want, which is a **write**. Then you must *read*. Two operations in opposite directions, in one transaction.

Repeated start

The sequence for reading one byte from a register is:

Start → device address + W → ACK → **register address** → ACK → **repeated START** → device address + R → ACK → **data** → NACK → **Stop**

The second start comes **without an intervening stop**. That is deliberate and it matters: a stop condition **releases the bus**, so another master could seize it and talk to a different device, after which your read would return the wrong device's data.

The repeated start holds the bus across the direction change, making the whole address-then-read sequence atomic.

That is the concept most people cannot explain when asked, and it is worth being able to: *why does an I^2C register read use a repeated start rather than a stop followed by a start?* Because the operation must not be interruptible.

5.1 Auto-increment

Devices with register addressing may offer **auto-increment**: after each byte, the internal register pointer advances. So one transaction can read six consecutive registers rather than six separate transactions.

Recall from Week 15 that SPI sensors do the same thing via bit 6 of the command byte. Here it is usually a configuration bit inside the device.

6 Deep dive 5: The peripheral, and a worked sensor

Book board vs your board

Book's F091RC: two peripherals, I2C1 and I2C2, with I2C1 offering more features.

Your F303VCT6: also has I2C1 and I2C2, and **I2C1 is wired to the on-board LSM303AGR accelerometer/magnetometer**. Your project plan puts it on **PB6 and PB7**. The book's example uses PB8 and PB9 with alternate function 1, so **do not copy its pin numbers**.

Note the pairing on your board: **accelerometer on I^2C, gyroscope on SPI**. Two different protocols, two sensors, one robot. That is not unusual in real designs.

Recall the four phases from Week 03. Clock gating via `RCC_APB1ENR`, then the pins to alternate function, then configuration, then enable.

Register	Field	Meaning
CR1	PE	Peripheral enable
CR1	TXIE, RXIE, NACKIE, STOPIE, TCIE, ADDRIE, ERRIE	Interrupt enables
CR2	SADD	Slave address
CR2	ADD10	7-bit (0) or 10-bit (1) addressing
CR2	RD_WRN	Transmit (0) or receive (1)
CR2	NACK	Send ACK (0) or NACK (1) after a received byte
CR2	NBYTES	**Number of bytes to transfer**
CR2	START, STOP	Generate the conditions
ISR	TC, BUSY, TXE, RXNE	Status
TDR, RDR	—	Transmit and receive data

Beware the trap

NBYTES is a design feature worth noticing. Unlike SPI, where you shift bytes until you stop, the STM32 I^{2}C peripheral **wants to know the transfer length in advance** so it can generate the ACK/NACK pattern and the stop condition automatically.

Set it wrong and the transaction ends early or hangs waiting for bytes that never come. And note the arithmetic in the book's write function: for a register write, NBYTES is data_len + 1, because **the register address counts as a transmitted byte**. Forgetting the +1 truncates your data by one byte.

6.1 The timing register

Beware the trap

I2Cx_TIMINGR configures the bus speed, and the book is refreshingly blunt: **computing its value is not trivial**. ST provides a separate application note and a tool purely for this.

It is not a simple divisor like the UART's BRR or SPI's BR, because I^{2}C requires you to specify setup times, hold times, and rise-time allowances separately, and those depend on the pull-up resistors and bus capacitance from Section 1.

Let CubeMX compute it. This is the one place in the course where the tooling is unambiguously the right answer, and the HAL will program TIMINGR correctly from the speed you select.

6.2 The write function

Listing 1: Book Listing 8.16, condensed. Note NBYTES = data + 1.

```
void I2C_WriteReg(uint8_t dev_adx, uint8_t reg_adx,
                  uint8_t *bufp, uint16_t data_len) {
    uint32_t tmp = 0;

    /* start + device address + write, length includes the register byte */
    MODIFY_FIELD(tmp, I2C_CR2_SADD,    dev_adx << 1);    /* the shift! */
    MODIFY_FIELD(tmp, I2C_CR2_RD_WRN, 0);                /* write      */
    MODIFY_FIELD(tmp, I2C_CR2_NBYTES, data_len + 1);     /* +1 for reg */
    MODIFY_FIELD(tmp, I2C_CR2_START,   1);
    I2C1->CR2 = tmp;
```

```
    while (I2C1->CR2 & I2C_CR2_START)    /* hardware clears START */
        ;

    I2C1->TXDR = reg_adx;                                    /* register address
        */
    while (!(I2C1->ISR & I2C_ISR_TXE))
        ;

    while (data_len--) {                                     /* the data */
        I2C1->TXDR = *bufp++;
        while (!(I2C1->ISR & I2C_ISR_TXE))
            ;
    }

    MODIFY_FIELD(I2C1->CR2, I2C_CR2_STOP, 1);
}
```

Note `dev_adx « 1`, which is the shift from Section 3 done explicitly. The book's code takes the 7-bit address and shifts it, so callers pass `0x6B` rather than `0xD6`. **HAL takes the shifted form.** Two conventions, and you must know which one your function uses.

6.3 The read function

Reading is "a little more complex", as the book puts it, because it is the repeated-start sequence from Section 4: send start with a *write* to convey the register address, then send **another** start with a *read*, then clock the data out, then stop.

6.4 The sensor

The book's example uses an LSM6DSL inertial module. Selected registers:

Register	Address	Purpose
`WHO_AM_I`	`0x0F`	Read-only ID, returns `0x6A`
`CTRL1_XL`	`0x10`	Accelerometer control 1
`CTRL3_C`	`0x12`	Control 3
`OUTX_L_XL`	`0x28`	X output, **least**-significant byte
`OUTX_H_XL`	`0x29`	X output, most-significant byte
`OUTY_L_XL`, `OUTY_H_XL`	`0x2A`, `0x2B`	Y output
`OUTZ_L_XL`, `OUTZ_H_XL`	`0x2C`, `0x2D`	Z output

Listing 2: Book Listing 8.19. Verify presence, then configure.

```
#define LSM6DSL_DEV_ADX          (0x6b)
#define LSM6DSL_REG_ADX_WHO_AM_I (0x0f)
#define LSM6DSL_WHO_AM_I_CODE    (0x6a)

void Init_Inertial_Sensor(void) {
    uint8_t data;

    I2C_ReadReg(LSM6DSL_DEV_ADX, LSM6DSL_REG_ADX_WHO_AM_I, &data, 1);
    if (data != LSM6DSL_WHO_AM_I_CODE)
        while (1);                       /* halt: sensor not found */

    data = 0x44;                         /* CTRL3_C: IF_INC=1 auto-increment,
                                                     BDU=1  block data update
                                            */
```

```
    I2C_WriteReg(LSM6DSL_DEV_ADX, LSM6DSL_REG_ADX_CTRL3_C, &data, 1);

    data = 0x40;                          /* CTRL1_XL: FS_XL=00 -> 2g range,
                                                       ODR_XL=0100 -> 104 Hz */
    I2C_WriteReg(LSM6DSL_DEV_ADX, LSM6DSL_REG_ADX_CTRL1_XL, &data, 1);
}
```

Three things in that initialisation are worth naming.

Check `WHO_AM_I` first and halt if wrong, exactly as in Week 15. Same technique, different bus.

`IF_INC` **enables auto-increment**, which is what makes the six-byte burst read possible. Without it you would need six separate transactions, and the book says plainly that would be slow.

`BDU`**, block data update**, is subtler and genuinely important.

Block data update, and why it exists

`BDU` prevents the sensor from updating the high byte of a reading while you are between reading its low byte and its high byte.

Recall Week 08's **partial update** problem: reading a 16-bit value as two separate bytes can give you the low byte of one sample and the high byte of the next, producing a number that was never measured.

`BDU` is the sensor solving that problem *in hardware, on its side of the bus*. It is the same concept as the critical section from Week 08 and the preload register from Week 13. **Turn it on.**

6.5 Using the data

The six output registers are consecutive and in LSB, MSB order, so with auto-increment enabled one six-byte read gets everything. Then scale by the sensitivity: at 0.001 g_n per bit, a reading of 1000 is one standard gravity, so `g_z` should be about 1000 with the board horizontal and face up.

Listing 3: Book Listing 8.19. Note the single-precision functions.

```
void convert_xyz_to_roll_pitch(void) {
    g_roll  = atan2f(g_y, g_z) * 180 / M_PI;
    g_pitch = atan2f(-g_x, sqrtf(g_y*g_y + g_z*g_z)) * 180 / M_PI;
}
```

Note `atan2f` and `sqrtf`, the **single**-precision variants, chosen deliberately over the slower but more accurate `atan2` and `sqrt`. Recall Week 10: on the book's Cortex-M0 every floating-point operation is a library call, and double precision costs roughly twice as much. On your Cortex-M4F the single-precision FPU handles `float` in hardware while `double` still goes to software, so **the choice matters even more on your board, not less**.

In the lab

This is your self-balancing robot's accelerometer path, essentially verbatim.
Your project plan reads the LSM303AGR over I2C1 on PB6/PB7 and expects `WHO_AM_I` at register `0x0F` to return `0x33`. Note that the register address is `0x0F` for the LSM303AGR, the I3G4250D, *and* the book's LSM6DSL. That is a strong convention across ST sensors, and a useful thing to assume when starting on a new one.
Your listed causes for a failed accelerometer read map exactly onto this week: **wrong address** (use `0x32` write, `0x33` read, which is Section 3's shift), **PB6/PB7 not set to I^2C alternate function** (which is Week 03), and **I^2C clock speed too high** (which is Section 1's bus-capacitance ceiling).

And the roll/pitch conversion above is the accelerometer half of your attitude estimate. The accelerometer gives you an absolute angle but is noisy under acceleration, and the gyroscope from Week 15 gives you a clean rate but drifts when integrated. Fusing them, whether with a complementary filter or a Kalman filter, is where your two buses meet. **Everything from Week 11's sampling through Week 13's PWM to these last three weeks converges on that one control loop.**

7 Where all of it lands: Lab 11

Lab 11 is the open-ended lab, and it is deliberately unguided: you must work out the angle estimation and the controller yourself, using everything from the previous sixteen weeks. So it is worth spelling out which week supplies which piece.

7.1 Task 1: the complementary filter

Fusing two imperfect sensors

Neither sensor alone gives you a usable angle.

The **accelerometer** measures gravity, so it gives a **stable long-term** tilt. But it also measures every other acceleration, so it is noisy the moment the robot moves.

The **gyroscope** measures angular rate cleanly, so integrating it gives excellent **short-term** angle changes. But any small bias in the rate accumulates without bound, so the integrated angle **drifts**.

A **complementary filter** takes a weighted average that trusts the gyroscope over the short term and lets the accelerometer slowly correct the drift:

$$\theta_n = 0.98 \times (\theta_{n-1} + \omega_x \cdot \Delta t) \; + \; 0.02 \times \theta_{acc}$$

where ω_x is the gyroscope rate in degrees per second and θ_{acc} is the accelerometer-derived tilt in degrees.

Read the two terms as two jobs. The 0.98 term is dead reckoning: take the previous angle and add how far you have rotated since. The 0.02 term is a slow pull back toward what gravity says. The gyroscope handles the fast motion, the accelerometer handles the truth, and the weights decide how quickly drift gets corrected against how much accelerometer noise leaks in.

Beware the trap

Δt must match your actual loop period, and this is where the whole course comes due.

The manual says it plainly: if your loop delays 100 ms, then $\Delta t = 0.1$. Use a Δt that does not match reality and your integrated gyroscope term is scaled wrongly, so your angle is systematically wrong and no amount of PID tuning will fix it.

Now recall Week 12's argument against software timing. If your loop period comes from `HAL_Delay` plus however long the sensor reads and UART prints happen to take, then Δt **is not actually constant**, it varies with every branch and every interrupt. Your filter is being fed a lie that changes size each iteration.

The fix is the whole point of Week 12: run the control loop from a **hardware timer interrupt** at a fixed rate, so Δt is exactly the timer period, every time. That is also the fix for the derivative term in the next section, and it is the single highest-value thing you can

do for stability.

7.2 Task 2: the PID controller

The three terms

Recall the error definition from Week 01: error = set point − measured value. A **PID** controller acts on it three ways.

Proportional: responds in proportion to the *current* error. Higher gain means faster response, and more overshoot or instability.
Integral: accumulates *past* error, which eliminates steady-state offset so the output eventually reaches the target even under a constant disturbance.
Derivative: uses the *rate of change* of error to predict where it is heading, which damps the response.

Your lab forbids a purely proportional controller, so you need at least PI or PD.

The pieces, and where each came from:

Stage	Mechanism	Covered in
Read accelerometer	I^2C, LSM303AGR	**Week 16**
Read gyroscope	SPI, I3G4250D	**Week 15**
Assemble 16-bit values	Byte order, `(hi « 8)	lo`	Week 06
Fixed sample interval	Timer interrupt, exact Δt	**Week 12**
Filter and PID maths	FPU, single-precision floats	Week 10, 11
Share data with the ISR	`volatile`, critical sections	**Week 08**
Drive the motors	PWM to the TB6612FNG	**Week 13**
Report what it is doing	UART, non-blocking	**Week 14**
Recover from a hang	Watchdog	Week 13

Beware the trap

Three failure modes that are really earlier weeks resurfacing, and worth checking before you touch your gains.

Oscillation that looks like bad tuning. If the loop period jitters, the derivative term is computed from an inconsistent Δt and produces noise rather than damping. Check that the loop is timer-driven before blaming K_d.

Slow drift in one direction. Recall Week 08's read-modify-write race: if an ISR accumulates sensor samples and the main loop reads-and-resets them without a critical section, you lose a sample every time the two collide, always in the same direction.

Motors respond backwards. The manual's own requirement is that positive angles turn both motors one way and negative angles the other. Recall Week 13: PWM *magnitude* sets speed and a separate direction pin sets sign, so a PID output that swings negative needs its sign split off before it reaches `CCRy`.

8 Deep dive 6: All three protocols, side by side

The comparison the exam wants. Recall Week 14's six problems.

	UART	SPI	I²C
Timing	Asynchronous	Synchronous	Synchronous
Wires (1 device)	2	4	**2**
Wires (n devices)	not possible	$3 + n$	**2, always**
Framing	Start/stop bits	Chip select	Start/stop conditions
Addressing	None	One select line each	**In the message**, 7-bit
Acknowledgment	None	None	**After every byte**
Error detection	Optional parity	None	ACK/NACK only
Speed	Slow	**Fast**, tens of MHz	100 k to 3.2 Mbit/s
Duplex	Independent both ways	Simultaneous exchange	Half duplex
Output type	Push-pull	Push-pull	**Open drain + pull-ups**
Devices	2	Many, pin-limited	**Many, 2 wires**
Key settings	Baud, bits, parity, stop	**CPOL, CPHA**, speed	Address, speed, `TIMINGR`

The three trades, in one sentence each

UART spends nothing and gets a slow, unaddressed, point-to-point link. Perfect for a debug console.

SPI spends *wires* to buy *speed* and *simplicity*, and gives up all feedback in exchange. Perfect for a fast sensor or a display.

I²C spends *speed* and *protocol complexity* to buy *a two-wire bus with addressing and acknowledgment*. Perfect for a handful of slow sensors sharing a board.

None dominates. Which is exactly why your board uses UART for the console, SPI for the gyroscope, and I²C for the accelerometer.

9 Tying it back, and the whole course

We asked whether several devices can share two wires with no select lines, and the answer works because of **open drain**: every device can only pull the line down, never up, so collisions are electrically harmless and the line is a wired-AND. That single circuit choice makes shared buses possible, and it costs you speed, because a pull-up resistor charging the bus capacitance is much slower than a transistor, which is why your maximum rate depends on your wiring rather than your chip.

With collisions made safe, the protocol puts **addresses into the message**, seven bits shifted left with the R/W bit in the LSB, which is the source of the `0x19` versus `0x32` confusion that catches everyone including you. And it adds the thing UART and SPI both lacked: an **acknowledgment after every byte**, which lets a master detect that nothing is listening and lets a bus be scanned. Reading a register needs a **repeated start** because you must write the register address and then read, without a stop in between, or another master could steal the bus in the gap.

And that closes the course. Look back at where it started: a hot plate whose bent metal strip overshot by a hundred degrees because it sensed the wrong thing, slowly, and could only

respond fully on or fully off. Every one of those faults now has a name and a peripheral. Sensing the right thing became the **ADC** in Week 11 and the sensors of these last two weeks. Sensing it quickly became **interrupts** in Weeks 7 and 8. Responding proportionally became **PWM** in Week 13. Doing all of it at once without falling over became **concurrency** in Weeks 4 and 5, and reporting what happened became **UART** in Week 14. Underneath it all, **GPIO** in Weeks 2 and 3 and the **core** in Weeks 6 and 10 are what let any of it touch a pin at all.

The hot plate, taken apart, is the entire syllabus. And the self-balancing robot is what you get when you put it back together with all sixteen weeks in it.

Cheat sheet: Week 16

Two wires and open drain

SDA data, **SCL** clock. Master/slave; master initiates everything.
Open drain: one transistor to ground per device, **no pull-up transistor**, plus an external pull-up resistor.
Send **0** = turn the transistor on. Send **1** = turn it off and let the resistor pull up.
Wired-AND: the line is 1 only if every device releases it. **Two devices disagreeing ⇒ line reads 0, no damage.**

Pull-ups (2.2–10 kΩ) are mandatory and are not in the MCU. On-board sensors have them fitted; a breadboard device does not. Smaller resistor = faster rise, more current wasted.
Speeds: **100 k** standard, **400 k** full, **1 M** fast, **3.2 M** high.
The real ceiling is set by bus capacitance — device count and bus length — not by the chip.

Message format

Start · **7-bit address** · **R/W** · **ACK** · **data byte(s)** · **Stop**. Optional **repeated start**.
Everything is a sequence of **conditions** (start, stop) and **transfers** (byte + ACK bit).

The ACK is what UART and SPI both lack. No ACK after the address ⇒ nothing is there ⇒ master sends stop.
This is why a bus scan is possible on I²C and not on SPI.

Who acknowledges: when *writing*, the **slave** ACKs each byte ("received"). When *reading*, the **master** ACKs ("send another") and sends a **NACK after the last byte** it wants. *Omitting the final NACK is a real bug.*

Addressing — the commonest bug

$$\text{first byte} = (\text{address} \ll 1) \mid \text{R/W}$$

Address `0x19` ⇒ **write** `0x32`**, read** `0x33`.
Datasheets disagree about which form they quote. Some give 7-bit, some the 8-bit write address, some the read address.
ST's HAL expects the shifted 8-bit form. The book's own functions do the shift internally and take the 7-bit form. **Know which convention your function uses.**
Symptom of a wrong address is identical to bad wiring or a dead device: no ACK.

Register access and the repeated start

Convention: the first data byte of a write is the **register address**.
Write: Start · dev+W · reg addr · data · Stop.
Read: Start · dev+W · reg addr · **repeated START** · dev+R · data · NACK · Stop.

Why a repeated start and not stop-then-start: a **stop releases the bus**, so another master could seize it between the address write and the read, and you would get the wrong device's data. **The repeated start keeps the transaction atomic.** *Learn this answer.*

Auto-increment advances the register pointer, so one transaction reads six consecutive registers instead of six transactions.

The peripheral

Clock via `RCC_APB1ENR`. Pins to alternate function (**yours: PB6/PB7 for I2C1**, not the book's PB8/PB9).
`CR1`: `PE` · `TXIE/RXIE/NACKIE/STOPIE/TCIE/ADDRIE/ERRIE`.
`CR2`: `SADD` · `ADD10` · `RD_WRN` · `NACK` · `NBYTES` · `START/STOP`.
`ISR`: `TC`, `BUSY`, `TXE`, `RXNE`. `TDR/RDR` for data.

`NBYTES` **must be set in advance** so the hardware can generate ACK/NACK and the stop. **For a register write,** `NBYTES` $=$ **data_len** $+$ **1** — the register address counts.
`TIMINGR` **is not a simple divisor** and its computation is non-trivial (ST publishes a tool for it). **Let CubeMX generate it.**

Sensor bring-up

Read `WHO_AM_I` **first and halt if wrong.** Register `0x0F` across LSM6DSL (`0x6A`), I3G4250D (`0xD3`), LSM303AGR (`0x33`).
`IF_INC` enables auto-increment ⇒ one six-byte burst instead of six transactions.
`BDU` **(block data update)** stops the sensor updating the high byte between your two reads **— it is the Week 08 partial-update problem solved in the sensor's hardware.** Turn it on.
Outputs are consecutive, **LSB then MSB**. Scale by sensitivity (0.001 g_n/bit ⇒ 1000 $=$ 1 g).
Use `atan2f`, `sqrtf` (single precision), not `atan2`/`sqrt`. Your M4F does `float` in hardware and `double` in software.

The three protocols — the exam comparison

UART: async · 2 wires · point-to-point only · start/stop framing · optional parity · **no ACK** · slow · independent TX/RX · push-pull.
SPI: sync · $3 + n$ **wires** · select-line addressing · **no ACK, no error detection** · **fastest** · simultaneous exchange · push-pull · **CPOL/CPHA**.
I²C: sync · **2 wires for any number of devices** · **addressing in the message** · **ACK every byte** · slower · half duplex · **open drain** $+$ **pull-ups**.

UART spends nothing. **SPI spends wires to buy speed. I²C spends speed to buy a shared bus.**

Likely exam prompts from this section

· **Compare UART, SPI, and I²C** across wires, addressing, speed, and acknowledgment. *Near certain.*
· Why does I²C need open-drain outputs and pull-up resistors?
· Given a 7-bit address, give the write and read bytes.
· Draw the message sequence for reading one byte from a slave register.
· **Why is a repeated start used rather than a stop followed by a start?**
· What limits the maximum speed of an I²C bus?
· What does the ACK bit indicate in each of its two uses?